

理論株価スクリーナー 要件定義書

対象バージョン: v2 (コア4条件・株規模表示・外部リンク実装済み) 更新日: 2025年 開発環境: VSCode / Python 3 / ブラウザ (Chrome / Safari)

目次

1. プロジェクト概要
 2. システム構成
 3. ファイル構成
 4. はっちゃん式 理論株価計算ロジック
 5. データ取得・キャッシュ仕様
 6. スクリーニング機能
 7. 詳細パネル機能
 8. チャート機能
 9. 企業価値コア4条件
 10. 株規模・投資難易度
 11. 外部リンク
 12. ウォッチリスト機能
 13. バックグラウンド収集機能
 14. UI/UX 仕様
 15. プロキシサーバー仕様
 16. 既知の制約・注意事項
 17. 今後の開発メモ
-

1. プロジェクト概要

目的

J-Quants V2 API を使い、**はっちゃん式理論株価**で日本株全銘柄をスクリーニングする中期投資支援ツール。

ターゲット

- J-Quants **Free** プランユーザー (データ遅延約90日・5リクエスト/分)
- ローカル環境で `proxy.py` を起動して使用

設計方針

- APIキーはメモリのみ保持 (ファイル保存・localStorage 保存なし)

- ・ 財務データ・銘柄マスターはローカルの `cache/` ディレクトリにJSONキャッシュ
- ・ フロントエンドはバニラJS（フレームワーク不使用）、Chart.js のみ外部依存

2. システム構成

```
[ブラウザ] index.html + js/*.js
  ↳ fetch (localhost:8765)
[proxy.py] Flask CORSプロキシ (127.0.0.1:8765)
  ↳ HTTPS
[J-Quants V2 API] api.jquants.com
  キャッシュ: cache/fins_cache.json / master_cache.json / price_cache.json
```

技術スタック

レイヤー	技術
フロントエンド	HTML / CSS / Vanilla JavaScript
チャート	Chart.js 4.4.0 (CDN)
バックエンド	Python 3 / Flask / requests
データソース	J-Quants V2 API

3. ファイル構成

```
/
├─ index.html      # HTML構造 + 追加CSSブロック + script src一覧
├─ style.css       # 全スタイル（ダークテーマ）
├─ proxy.py        # Flask CORSプロキシ
├─ cache/          # 自動生成キャッシュ（gitignore推奨）
│   ├─ fins_cache.json
│   ├─ master_cache.json
│   └─ price_cache.json
├─ js/             # 機能別分割JS（読み込み順 = 依存順）
│   ├─ theory.js   # 理論株価計算・フォーマッタ（純粋関数・DOM依存なし）
│   ├─ state.js    # 全グローバル変数の一元管理
│   ├─ api.js      # 日付ユーティリティ・株価取得ヘルパー
│   ├─ chart.js    # 株式分割調整・チャート描画
│   ├─ table.js    # テーブル描画・コア条件バッジ・詳細開閉
│   └─ watchlist.js # ウォッチリスト管理・UI
```

```
└─ sort.js          # マルチキーソート
└─ background.js    # バックグラウンド財務データ収集 + 条件自動計算
└─ connect.js       # 接続・切断・フィルタ読み込み
└─ screening.js     # スクリーニング計算・実行
└─ detail.js        # loadDetail・renderDetail (株規模・外部リンク含む)
└─ conditions.js    # コア4条件計算・BG軽量計算・詳細パネル描画
└─ app.js           # ツールチップ・ペインリサイザー (最後に読み込み)
```

読み込み順は `index.html` 末尾の `<script src>` の並び順で管理。`conditions.js` は必ず `detail.js` の直後に読み込む (`updateRowBadge` は `table.js` で定義)。

グローバル変数一覧 (`state.js`)

変数	型	用途
<code>PROXY</code>	string	プロキシURL (<code>http://localhost:8765</code>)
<code>masterCache</code>	array null	銘柄マスターデータ
<code>priceCache</code>	object	<code>{date: {code: quote}}</code> の株価キャッシュ
<code>finsCache</code>	object	<code>{code: [stmt,...]}</code> の財務キャッシュ
<code>condCache</code>	object	<code>{code: condResult}</code> のコア4条件キャッシュ
<code>screenAbortCtrl</code>	AbortController null	スクリーニング中断用
<code>lastResults</code>	array	直近のスクリーニング結果 (ソート再適用に使用)
<code>screeningDate</code>	string null	スクリーニング基準日
<code>screeningPrices</code>	object	スクリーニング基準日の株価マップ
<code>activeCode</code>	string null	詳細表示中の証券コード
<code>chart</code>	Chart null	Chart.js インスタンス
<code>currentDetailData</code>	object null	チャートスケール切替用 (詳細データ保持)
<code>chartScaleMode</code>	string	<code>'all'</code> or <code>'price'</code>
<code>sortKeys</code>	array	<code>[[{key,dir}]]</code> マルチキーソート設定 (localStorage永続化)
<code>watchlist</code>	array	<code>[[{code,name,market,sector}]]</code> (localStorage永続化)
<code>bgEnabled</code>	boolean	BG収集ON/OFF (localStorage永続化)

変数	型	用途
<code>bgRunning</code>	boolean	BG収集実行中フラグ
<code>bgIndex</code>	number	BG収集の次の銘柄インデックス
<code>bgInitialized</code>	boolean	BG再開位置の初期化済みフラグ

4. はっしゅん式 理論株価計算ロジック

4.1 中核関数 (`js/theory.js`)

理論株価 = (資産価値 + 事業価値) × (1 - リスク率)
 上限株価 = 資産価値 + 事業価値 × 2

変数	計算式
資産価値 (av)	<code>BPS × eqRate(自己資本比率)</code>
事業価値 (bv)	<code>15 × 予想EPS × min(ROA, 0.3) × 10 × lev(自己資本比率)</code>
eqRate	<code>≥80%→0.8 / ≥67%→0.75 / ≥50%→0.7 / ≥33%→0.65 / ≥10%→0.6 / else→0.5</code>
lev	<code>1 / min(1, max(0.66, EqAR + 0.33))</code>
riskRate	<code>PBR≥0.5→0 / ≥0.41→0.2 / ≥0.34→0.33 / ≥0.25→0.5 / ≥0.21→0.66 / ≥0.04→線形0.75~0.95</code>

4.2 α値 (割安度)

$\alpha \text{値}(\%) = (\text{理論株価} - \text{株価}) / \text{株価} \times 100$

α値	水準ラベル	色
> 100%	超割安	緑  #10b981
50~100%	割安	 #34d399
10~50%	準割安	 #6ee7b7
-10~10%	適正	グレー  #94a3b8
-30~-10%	準割高	黄  #fbbf24
-50~-30%	割高	橙  #f97316
< -50%	超割高	赤  #ef4444

4.3 予想EPSの算出ルール（重要）

$$\text{予想EPS} = fOp \times 0.7 / \text{発行株数}$$

fOp（利益ベース）の選択優先順位：

基準レコード = `stmts.filter(BPS > 0)` の最新レコード（FY・四半期どちらも対象）

- ① 基準レコードが FY かつ `OdP > 0`
 - `fOp = OdP`（当期実績を最優先）
 - ※ 旧四半期のF0dP（予想）で上書きしない
- ② 基準レコードが四半期、または FY で `OdP = 0`
 - 最新から遡って `F0dP > 0` を検索
 - `fOp = F0dP`（当期年間予想）
- ③ `fOp = 0` の場合
 - `fEps = annOpD × 0.7 / shares`（年率換算実績で代替）

背景: FY確定後も古い四半期 F0dP が残るため遡り検索では旧予想を拾ってしまう。FY実績を最優先することで上段・下段・スクリーニングを一貫させる。

4.4 ROAの年率換算（四半期対応）

CurPerType	係数	計算式
1Q	×4	<code>annualOp = OdP × 4</code>
2Q	×2	<code>annualOp = OdP × 2</code>

CurPerType	係数	計算式
3Q	×4/3	<code>annualOp = OdP × 4/3</code>
FY	×1	<code>annualOp = OdP</code>

`ROA = annualOp / TA`

4.5 上限株価の計算（詳細上段）

詳細パネルの上段では理論株価と上限株価を別系統で計算する。

```

javascript

const opForUpper = fOp > 0 ? fOp : annOpD; // FY実績 or 四半期年率換算
const fEpsForUpper = opForUpper × 0.7 / shares;
上限株価 = calcTheory(bps, fEpsForUpper, roaRaw, eq, pbr).upper

```

4.6 基準レコードの選択（統一ルール）

スクリーニング・詳細上段・条件計算すべてで共通：

```

javascript

const validStmts = stmts.filter(s => parseFloat(s.BPS || 0) > 0);
const baseRecord = validStmts[validStmts.length - 1]; // 最新（FY・四半期間わず）

```

FY限定ではなく四半期も含める。BPSは分割調整なしの原数値。

5. データ取得・キャッシュ仕様

5.1 J-Quants V2 API エンドポイント

用途	エンドポイント	レスポンスキー
銘柄マスター	<code>GET /v2/equities/master</code>	<code>data</code> (<code>info</code> , <code>items</code> 等もフォーマルバック)
株価（日次 bulk）	<code>GET /v2/equities/bars/daily?</code> <code>date=YYYYMMDD</code>	<code>data</code>
株価（個別履 歴）	<code>GET /v2/equities/bars/daily?</code> <code>code=XXXXX</code>	<code>data</code>
財務サマリー	<code>GET /v2/fins/summary?code=XXXXX</code>	<code>data</code>

5.2 プロキシエンドポイント (localhost:8765)

パス	説明
<code>POST /proxy/connect</code>	APIキー認証・接続 (トヨタ86970で疎通確認)
<code>POST /proxy/disconnect</code>	切断 (財務キャッシュ保存)
<code>GET /proxy/status</code>	接続状態確認
<code>GET /proxy/master</code>	銘柄マスター (キャッシュ優先)
<code>GET /proxy/prices?date=YYYYMMDD</code>	全銘柄株価・bulk (日次キャッシュ)
<code>GET /proxy/prices?code=XXXXX</code>	個別銘柄株価履歴 (キャッシュなし)
<code>GET /proxy/fins?code=XXXXX</code>	財務サマリー (キャッシュ優先)
<code>GET /proxy/cache_info</code>	キャッシュ状況確認
<code>POST /proxy/cache_clear</code>	キャッシュ全削除

5.3 レート制限 (Free プラン対応)

- プロキシ側スロットリング: **14秒間隔** (`RATE_INTERVAL = 14.0`)
- バックグラウンド収集: **13秒間隔** (`BG_INTERVAL_MS = 13000`)
- スクリーニング中はBG収集を自動スキップ

5.4 株価基準日の決定フロー

- トヨタ(72030)の個別履歴で最新日を特定 → `probeDate`
- `probeDate`でbulk取得を試行 (最大5日前まで後退)
- 失敗時フォールバック:
`localStorage`保存日 → `latestDateStart()` (90日前の直近営業日)

5.5 日付ユーティリティ (`js/api.js`)

関数	内容
<code>prevTradingDay(dateStr)</code>	前営業日 (土日スキップ)
<code>nextTradingDay(dateStr)</code>	翌営業日 (土日スキップ)
<code>latestDate()</code>	直近営業日 (3日前基準・Freeプラン遅延考慮)

関数	内容
latestDateStart()	90日前の直近営業日（初回フォールバック用）

6. スクリーニング機能

6.1 フィルタ条件

フィルタ	ID	デフォルト	説明
市場	f-market	すべて	東証プライム/スタンダード/グロース等
業種	f-sector	すべて	東証33業種分類
α値 下限	f-mina	10%	割安度の最小値
α値 上限	f-maxa	200%	異常値除外
自己資本比率 下限	f-eq	30%	財務健全性フィルタ
ROA 下限	f-roa	0%	収益性フィルタ
配当利回り 下限	f-div	0%	高配当フィルタ
コア条件 適合数	f-cond	指定なし	0～4件。詳細表示またはBG計算済みの銘柄のみ有効
バリュートラップ除外	f-notrap	ON	PBR<0.3 かつ α>100% を除外

6.2 2段階表示（Phase1/2）

Phase1: finsCache にキャッシュ済みの銘柄を即座に表示
Phase2: 未取得銘柄を順次API取得しながら20件ごとに更新
完了後: 最終確定結果を表示

フィルタ変更・中断ボタンで AbortController により即座に中断。

6.3 結果テーブル列（左→右）

コード / 銘柄名 / 株価 / 理論株価 / α値 / 水準 / 上限株価 / 資産価値 / 事業価値 / BPS / 予想EPS / ROA% / 自己資本比率% / PBR / 配当利回り% / 業種 / コア条件

6.4 コア条件列の表示仕様

4つのドットのみ表示（N/4スコア表記なし）。

状態	表示	説明
未計算	 (暗グレー  #334155)	詳細を開くと計算
BG計算済み	 (①②暗、③④色付き)	fins dataで計算可能な条件のみ
詳細表示後	 (全4条件、色付き)	完全計算済み

- (緑  #10b981) = 適合
- (グレー  #475569) = 非適合
- (暗  #334155) = 未計算
- 各ドットに `title` 属性で条件名・判定結果を表示

6.5 マルチキーソート (`js/sort.js`)

- 最大3キーの優先順位ソート
- `condScore` キーは `condCache[code]?.score` を動的参照 (スクリーニング時のスナップショット非依存)
- `condScore` がアクティブソートキーの場合、`updateRowBadge` 後に全体を再ソート・再描画
- ソート設定は `localStorage` に永続化

7. 詳細パネル機能

7.1 パネル構成 (上から順)

```

detail-header (コード・銘柄名・市場/業種・閉じるボタン・ウォッチ登録ボタン)
ext-links (外部リンクボタン群)
vt-banner (バリュートラップ警告: 条件該当時のみ)
metric-grid (主要指標グリッド)
size-wrap (株規模・投資難易度)
bar-wrap (株価水準イメージバー)
chart-wrap (株価チャート)
hist-wrap (財務履歴テーブル)
cond-block (コア4条件: conditions.jsが描画)

```

7.2 上段メトリクス (最新レコードベースで統一)

基準レコード: `stmts.filter(BPS > 0)` の最後の要素 (FY・四半期どちらも対象)

項目	算出元	備考
株価	スクリーニング価格優先、なければ最新終値	
PBR	株価 ÷ BPS	
α値・水準	<code>calcAlpha</code> / <code>levelOf</code>	
理論株価	<code>calcTheory(bps, fEpsRaw, roaRaw, eq, pbr)</code>	
上限株価	<code>calcTheory(bps, fEpsForUpper, roaRaw, eq, pbr).upper</code>	fEpsForUpperは別計算
ROA	annOpD / TA	四半期は年率換算済み
自己資本比率	ls.EqAR	
BPS	ls.BPS	
予想EPS	fOp × 0.7 / shares	FY実績優先ロジック
配当（年間）	FDivAnn / DivAnn 等を最新から遡って取得	
配当4%ライン	年間配当 ÷ 0.04	

7.3 バリュートラップ警告バナー

表示条件: PBR < 0.3 かつ α値 > 100%

赤色バナーをメトリクスグリッド直前に表示。ホバーでPBR値・リスク説明。

7.4 財務履歴テーブル

- **FY（通期）**：直近8期、**四半期（1Q/2Q/3Q）**：直近6期
- 期末日順にソート・表示
- 各行は**その期のデータのみ**で独立計算
- `div`（配当額・分割調整済み）・`div4`（配当4%株価）フィールドを含む
- 株式分割補正: 期末日より後の分割係数で除算（`÷N` 表記付き）

EPS 注記の意味:

注記	意味
(なし)	FY・FOdP>0 → 当期予想ベース
実	FY・FOdP=0 → 実績OdP
予	四半期・FOdP>0 → 当期年間予想
推	四半期・FOdP=0 → 年率換算実績で推計

8. チャート機能

8.1 基本仕様

- ・ **期間:** 直近504営業日（約2年）、`getSplitAdjustedHistory(quotes).slice(-504)`
- ・ **データ:** 株式分割調整済み終値
- ・ **Y軸モード:** 全体（理論・上限含む） / 株価基準（株価前後のみ）

8.2 株式分割調整の優先順位

1. `AdjustmentClose` が有効 → そのまま使用
2. `AdjustmentFactor` が実質機能（1以外存在） → `Close × Factor`
3. フォールバック → 終値の急変（1.8倍以上）から分割比を自動検出・逆算

8.3 理論株価ラインの時系列

チャート用データ: `chartHist` (全BPSレコード・スライス制限なし)
 表示用データ: `hist` (FY8期 + 四半期6期)

各日付の値 = `effectivePeriod` (開示日) 以降で最新の財務レコードの理論株価
`effectivePeriod` = `DisclosureDate` 優先、なければ期末日推定

8.4 財務データ不足期間のグレーライン

最古 `chartHist` レコードの `effectivePeriod` より前の期間:

- グレー破線: 理論株価(参考)・上限株価(参考)
- 色: `#64748b` / `#94a3b8`、線幅1、`dash=[4,4]`

確定データ期間:

- 黄系実線: 理論株価 (`#eab308`)・上限株価 (`#f59e0b`)
- 線幅1.5、`dash=[6,3]/[3,3]`

8.5 配当利回りライン（配当銘柄のみ）

- 右軸（0～8%）に赤色破線（#ef4444）
- $\text{利回り} = \text{年間配当} / \text{終値} \times 100$

9. 企業価値コア4条件

9.1 概要

中長期投資に適した銘柄の特性を4条件で評価。10年間条件を2年データに換算して判定。

9.2 条件詳細

① 正相関度（corr）

必要データ: 個別株価履歴（詳細表示時に自動取得）

チェック	閾値	方法
全期間相関	≥ 50%	Pearson相関（closes × theoryLine、2年分・最低10点）
1年相関	≥ 50%	直近252日のPearson相関
理論株価傾向	上昇	$\text{chartHist.last.theory} \geq \text{chartHist.first.theory}$

全3項目が満たされた場合 ok = true

② 割安修正（disc）

必要データ: 個別株価履歴（詳細表示時に自動取得）

チェック	閾値	方法
全平均α値	≥ 0%	hist各期の開示日近傍株価でα値を計算し平均（最低2期）
最新α値	≥ 0%	現在のスクリーニングα値
改善傾向	最新 < 平均	割安が解消されつつある（株価が理論株価に近づく）

全3項目が満たされた場合 ok = true

③ 持続成長（growth）

必要データ: fins dataのみ → BG収集時に自動計算可能

チェック	閾値	方法
理論株価 純増回数	≥ +2	hist理論株価の時系列増減を集計（増加+1・減少-1）
最高値比	≥ 85%	最新theory / max(hist.theory)

2年換算: 10年+8 → 閾値+2

④ 配当成長 (div)

必要データ: fins dataのみ → BG収集時に自動計算可能

チェック	閾値	方法
配当 純増回数	≥ +1	FY履歴の配当額増減を集計（最低2件のFYデータ）
配当4%株価 最高値比	≥ 85%	最新div4 / max(hist.div4)

2年換算: 10年+8 → 閾値+1 無配当銘柄: {noDiv:true, ok:false} / FYデータ2件未満: null

9.3 計算タイミング (2段階)

【BG収集時】 fins dataのみ・追加API不要
 → ③持続成長・④配当成長 を buildLightHist で計算
 → ①② は null（詳細を開くと計算）
 → condCache[code] = {..., bgOnly: true}

【詳細表示時】 個別株価履歴も使用
 → ①②③④ 全条件を computeConditions で完全計算
 → condCache[code] = {..., bgOnly: false} (bgOnly=trueを上書き)

9.4 condCache の構造

javascript

```
condCache[code] = {
  corr:    {fullCorr, oneYearCorr, theoryGrowth, ok} | null,
  disc:    {avgAlpha, latestAlpha, ok} | null,
  growth:  {netMoves, peakRatio, total, ok} | null,
  div:     {netMoves, div4PeakRatio, dataPoints, ok}
           | {noDiv:true, ok:false}
           | null,
  score:   0~4,           // ok=true の数
  bgOnly:  boolean,       // BG計算済み（詳細表示で完全版に上書き）
  computed: true
}
```

9.5 スクリーニングリスト表示

- テーブル最右列にドット4つ（N/4スコア表記なし）
- 各ドットの `title` 属性に条件名・判定を記載
- `updateRowBadge(code)`: `table.js` で定義。`conditions.js` に重複定義しない
- `condScore` ソートは `condCache` を動的参照し BG更新後も即反映

9.6 詳細パネル表示（`#cond-block`）

- BG計算済み（`bgOnly:true`）: ①②を「詳細を開くと計算（株価履歴が必要）」🔍で表示
- 完全計算後: 全4条件の数値詳細（相関値・α値・純増数・最高値比）を表示
- 各条件アイテムにホバーで詳細説明ツールチップ

10. 株規模・投資難易度

詳細パネルのメトリクスグリッド直後に `.size-wrap` セクションとして表示。実装: `js/detail.js` の `classifySize` / `liquidityRisk` / `pbrZone` / `roaStability`。

`marketCap = price × shares` (shares = 1s.ShOutFY)

時価総額による規模分類（`classifySize`）：

時価総額	ラベル	カラー	投資適性
≥ 1兆円	大型株	青 #3b82f6	中長期向き・安定
≥ 300億円	中型株	緑 #10b981	中長期向き・標準
≥ 50億円	小型株	黄 #f59e0b	上級者向き・注意
< 50億円	超小型株	赤 #ef4444	上級者専用・高リスク

流動性リスク (liquidityRisk) :

時価総額	ラベル
≧ 1000億円	流動性 十分
≧ 300億円	流動性 普通
≧ 50億円	流動性 注意
< 50億円	流動性 高リスク

PBR安心度 (pbrZone) :

PBR	ラベル
< 0.3	超割安注意 (バリュートラップリスク)
0.3~0.5	要観察
0.5~3.0	標準圏 ✓
3.0~5.0	やや割高
> 5.0	割高圏

業績安定度 (roaStability) : ROAの標準偏差 (過去N期)

σ	ラベル
< 1.5%	高安定 ✓
1.5~4%	標準
> 4%	変動大
N < 3	データ不足

11. 外部リンク

詳細パネルのヘッダー直下に `.ext-links` として横並び表示。

コード変換: J-Quantsの5桁コード → 4桁 (`code.slice(0, 4)`)

例: `72030` → `7203` (末尾の0は株種区分)

サービス	URL パターン	内容
IR Bank	<code>https://irbank.net/{code4}/results</code>	過去10年財務（無料・最詳細）
株探	<code>https://kabutan.jp/stock/?code={code4}</code>	決算・ニュース・チャート
Yahoo!ファイナンス	<code>https://finance.yahoo.co.jp/quote/{code4}.T</code>	株価・指標・掲示板
みんかぶ	<code>https://minkabu.jp/stock/{code4}</code>	予想・目標 株価・口コミ
適時開示 (TDnet)	<code>https://www.release.tdnet.info/inbs/I_main_00.html?code={code4}</code>	決算短信原本

12. ウォッチリスト機能

- 登録銘柄をリスト表示・スクリーニング結果に反映
- ドラッグ&ドロップで並べ替え
- 証券コード直接入力で追加（masterCache から名称自動補完）
- 登録情報は `localStorage` に永続化（コード・銘柄名・市場・業種）
- ウォッチリスト表示中はフィルタなしで全登録銘柄をスクリーニング

13. バックグラウンド収集機能

動作フロー

- 接続後・masterCache 取得済み → BG toggle ONで自動開始
- 13秒間隔でループ（スクリーニング中はスキップ）
- finsCache に未登録の銘柄を1件取得（proxy経由・14秒スロットリング）
- 取得後、即座に BG条件計算を実行：


```
computeConditionsBG(code)
  → buildLightHist(stmts) で軽量hist構築（split調整なし）
  → ③持続成長・④配当成長 を評価
  → condCache[code] = {growth, div, score, bgOnly:true}
```


→ `updateRowBadge(code)` でテーブル行をリアルタイム更新

5. 全銘柄収集完了でBG停止

buildLightHist (BG専用・軽量版)

- split調整なし (精度より速度優先)
- 詳細表示時の `buildHistArray` とは別実装
- `conditions.js` に実装 (`computeConditionsBG` と同ファイル)

重要なルール

- `condCache[code].bgOnly = true` のものは詳細表示時の完全版で上書き
- 詳細で完全計算済み (`bgOnly = false`) なら BG計算は上書きしない
- 次回再開コードを `localStorage` (`screener_bg_next_code`) に保存

14. UI/UX 仕様

14.1 レイアウト



14.2 テーマ・カラー

変数	値	用途
<code>--accent</code>	 <code>#3b82f6</code>	青・株価・アクセント
<code>--gold</code>	 <code>#eab308</code>	金・理論株価

変数	値	用途
--amber	 #f59e0b	琥珀・上限株価
--green	 #10b981	緑・正值・適合
--red	 #ef4444	赤・負値・警告
--muted	 #94a3b8	グレー・補足情報
--border	—	ボーダー

背景:  #0d1120、テキスト:  #e2e8f0

14.3 ツールチップ

- `data-tip` 属性に説明文を記載
- `js/app.js` のIIFEで実装（マウス追従・画面端自動反転・スクロール/クリックで消去）

14.4 ペインリサイザー

- 中央の仕切りをドラッグで詳細パネル幅を調整（280～900px）
- `localStorage` (`screeners_detail_width`) に永続化

14.5 新規追加CSS (`index.html` の `<style>` ブロック)

CSS

```

/* バリュートラップ警告バナー */
.vt-banner { ... }

/* 外部リンク */
.ext-links { ... }
.ext-link { ... }

/* 株規模・リスク */
.size-wrap { ... }
.size-grid { grid-template-columns: repeat(2, 1fr); }
.size-item { ... }
.size-badge { ... }

/* コア4条件 */
.cond-wrap { ... }
.cond-header { ... }
.cond-score { letter-spacing: 2px; }
.cond-grid { grid-template-columns: repeat(2, 1fr); }
.cond-item { border-left: 2px solid; }
.cond-item.ok { border-left-color: #10b981; }
.cond-item.ng { border-left-color: #334155; }
.cond-item.pending { opacity: .7; }

/* テーブルのコア条件列 */
.td-cond { text-align: center; white-space: nowrap; }

```

15. プロキシサーバー仕様

proxy.py 概要

- Flask ベースのローカル CORS プロキシ
- ポート: (8765) ((127.0.0.1) のみバインド)
- APIキーはメモリのみ ((mem["api_key"])) ・ ファイル保存なし

キャッシュ仕様

キャッシュ	ファイル	形式	保存タイミング
財務データ	cache/fins_cache.json	{code:[stmt,...]}	20件ごと・切断時・終了時
銘柄マスター	cache/master_cache.json	[{Code,CoName,...}]	取得時即保存

キャッシュ	ファイル	形式	保存タイミング
株価（日次 bulk）	cache/price_cache.json	{date:{code:quote}}	取得時即保存

起動時: 旧形式（dict単体）→ 新形式（list）に自動変換。

16. 既知の制約・注意事項

API・データ制約

制約	内容
データ遅延	Free プランは約90日遅延
財務履歴深度	API が返す財務データは直近約2年分のみの場合あり
チャート前半グレー	最古財務レコード以前は理論株価算出不可（グレー参考表示）
レート制限	5リクエスト/分（超過すると429エラー）

計算上の注意

- ・ 四半期 BPS: 一部銘柄では四半期報告に BPS が含まれない→スキップされる
- ・ 株式分割検出: AdjustmentClose がいない場合は終値変動から推定（精度は参考値）
- ・ BG条件計算: split調整なしの軽量版。詳細表示で完全版に上書きされる
- ・ condScore ソート: condCache を動的参照するため BG更新後もソートが有効

ブラウザ制約

- ・ Safari 対応: var 宣言を使用 (let のブロックスコープ起因のエラーを回避)
- ・ localStorage 使用（プライベートブラウズでは動作が異なる場合あり）

関数の重複定義禁止

- ・ updateRowBadge は table.js にのみ定義する
- ・ conditions.js に同名関数を定義すると後から読み込まれて上書きし、2行表示バグが発生する（過去のバグ）

17. 今後の開発メモ

検討中・未実装

- ☐ ①正相関度・②割安修正のBG計算（個別株価履歴のAPI取得が必要・28秒/銘柄）
- ☐ チャート期間切替（1年 / 3年 / 5年）
- ☐ CSV エクスポート（スクリーニング結果）
- ☐ フィルタプリセットの保存・呼び出し
- ☐ 為替（ドル円）との相関分析（外部API必要）
- ☐ 移動平均線のチャートへの追加
- ☐ IR カレンダー表示（決算発表予定日）

ファイル別変更ガイド

変更内容	対象ファイル
理論株価の計算式	js/theory.js
フィルタ条件の追加	js/screening.js → calcScreenResults + index.html → フィルタUI
テーブル列の追加	js/table.js → renderTable + index.html → <thead>
コア4条件の閾値	js/conditions.js → computeConditions / computeConditionsBG
BG計算の対象条件	js/conditions.js → computeConditionsBG
詳細パネルの表示項目	js/detail.js → renderDetail
株規模・リスク分類の 閾値	js/detail.js → classifySize / liquidityRisk / pbrZone / roaStability
外部リンクの追加	js/detail.js → renderDetail の .ext-links セクション
チャートの見た目	js/chart.js → buildChart
API エンドポイント変 更	js/api.js + proxy.py
新しいグローバル変数	js/state.js
BG収集のロジック	js/background.js → bgFetchNext
ツールチップ・リサイ ザー	js/app.js